

Оглавление

1	Аналитический раздел	6
1.1	Анализ предметной области	6
1.2	Сравнительный анализ существующих систем	6
1.3	Требования к проектируемой системе	7
1.3.1	Функциональные требования	7
1.3.2	Нефункциональные требования	7
1.3.3	Ограничения	8
1.4	Метрики эффективности и правила их расчёта	8
1.5	Сущности предметной области, роли пользователей и варианты использования	9
1.5.1	Сущности и связи	9
1.5.2	Роли пользователей и варианты использования	10
1.6	Выбор модели данных	11
1.7	Выводы по аналитическому разделу	12
2	Конструкторский раздел	14
2.1	Логическое проектирование базы данных	14
2.1.1	Соответствие нормальным формам	19
2.1.2	Проектирование представлений	19
2.1.3	Проектирование индексов	20
2.2	Проектирование ограничений целостности	20
2.3	Проектирование алгоритмов обработки данных	21
2.3.1	Алгоритмы расчёта метрик игроков	21
2.3.2	Алгоритм агрегирования командной статистики	21
2.3.3	Алгоритм массового обновления агрегатов	22
2.4	Проектирование ролевой модели доступа	22
2.5	Проектирование программного обеспечения и кэширования	23
2.5.1	Архитектура приложения	23
2.5.2	Спецификация REST API	23
2.5.3	Кэширование результатов запросов	25
2.6	Выводы по конструкторскому разделу	26
3	Технологический раздел	27

3.1	Выбор и обоснование средств реализации	27
3.2	Физическая реализация базы данных	27
3.2.1	Создание таблиц и ограничений	27
3.2.2	Реализация индексов	28
3.2.3	Загрузка тестовых данных	29
3.3	Реализация серверной логики	29
3.3.1	Хранимые функции расчёта метрик	29
3.3.2	Бизнес-триггер	30
3.4	Реализация ролевой модели на уровне БД	31
3.5	Реализация интерфейса доступа к данным	32
3.5.1	Серверная часть	32
3.5.2	Клиентская часть	32
3.6	Тестирование системы	34
3.6.1	Тестирование хранимой функции	34
3.6.2	Верификация правильности расчёта метрик	34
3.7	Выводы по технологическому разделу	34
4	Исследовательский раздел	35
4.1	Цель и методика исследования	35
4.2	Влияние объёма данных и стратегий индексации	35
4.2.1	Влияние объёма данных	35
4.2.2	Влияние стратегий индексации	36
4.3	Исследование эффективности кэширования	37
4.4	Исследование производительности при параллельной нагрузке	37
4.5	Анализ результатов и рекомендации	38
4.6	Выводы по исследовательскому разделу	39
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	40

1 Аналитический раздел

1.1 Анализ предметной области

Национальная баскетбольная ассоциация (NBA) является источником одного из наиболее детализированных массивов спортивной статистики среди крупнейших мировых профессиональных лиг [1]. Для каждого матча фиксируются итоговый счёт, составы команд и числовые показатели по каждому игроку: очки, подборы, передачи, блок-шоты, перехваты и иные. На этой основе можно строить рейтинги, сопоставлять сезоны и выявлять тенденции развития игры.

Современные аналитические подходы требуют использования не только базовых статистических показателей, но и производных метрик эффективности: истинного процента бросков TS%, эффективного процента с игры eFG%, доли владений в атаке USG%, рейтинга эффективности игрока PER, показателя вклада игрока (Box Plus/Minus, BPM) [2–5]. Расчёт этих показателей требует хранения непротиворечивых первичных данных по матчам, их агрегирования по сезонам и применения фиксированных правил вычисления.

Разработка специализированной базы данных обусловлена тремя факторами: необходимостью гарантировать логическую целостность данных — соответствие суммы индивидуальных очков командному счёту, потребностью в оптимизированном хранении для ресурсоёмких агрегатных вычислений и требованием поддерживать высокую нагрузку при параллельном доступе многих клиентов к аналитическим данным.

1.2 Сравнительный анализ существующих систем

Перед проектированием собственного решения проведём анализ функциональных возможностей наиболее распространённых публичных систем сбора и анализа статистики NBA: официального портала NBA Stats [1], справочной платформы Basketball-Reference [3] и аналитического раздела ESPN [6].

Таблица 1.1 — Сравнительный анализ существующих систем по функциональным возможностям

Критерий	NBA.com Stats	Basketball-Reference	ESPN Stats
Просмотр статистики матчей	+	+	+
Фильтрация по сезону, позиции, команде	частично	+	частично
Расчёт TS%, eFG%, PER, BPM	+(формулы скрыты)	+(формулы открыты [4, 5])	частично
Произвольные аналитические запросы	–	–	–
Сравнение игроков	частично	–	+
Ролевое разграничение доступа	–	–	–
Локальное хранение, воспроизводимость расчётов	–	–	–

Анализ показывает, что NBA.com Stats и ESPN предоставляют данные через готовый пользовательский интерфейс с непрозрачными формулами расчёта метрик. Basketball-Reference публикует описание своих формул [4, 5], однако не предоставляет открытого API для произвольных аналитических запросов. Ни одна из систем не поддерживает ролевого разграничения доступа и воспроизводимого локального хранения данных. Выявленные ограничения определяют требования к проектируемой системе, сформулированные в следующем разделе.

1.3 Требования к проектируемой системе

1.3.1 Функциональные требования

- 1) хранение справочных данных: игровые позиции, сезоны, клубы NBA;
- 2) хранение профилей игроков: персональные и антропометрические данные, позиция, сведения о драфте;
- 3) хранение данных о матчах: сезон, команды-участники, дата, итоговый счёт, статус;
- 4) хранение детализированной статистики игрока в матче: очки, подборы, передачи, броски с игры и штрафные, потери, фолы, время на площадке;
- 5) хранение агрегированной статистики игрока за сезон: средние значения, процентные показатели, расчётные метрики;
- 6) хранение истории выступлений игрока за клубы по сезонам;
- 7) расчёт командной статистики в рамках матча и сезона на основе индивидуальных показателей игроков;
- 8) автоматизированный расчёт метрик эффективности (eFG%, TS%, USG%, PER, BPM) на основе первичных фактов матчей;
- 9) предоставление сводных рейтингов игроков и турнирных таблиц команд.

1.3.2 Нефункциональные требования

- 1) производительность: время отклика на аналитические запросы чтения (агрегация за сезон) не должно превышать 200 мс;
- 2) объём данных: система должна выдерживать нагрузку не менее 100 000 записей в таблице фактов (соответствует статистике за 5 регулярных сезонов);
- 3) целостность: база данных должна соответствовать требованиям ACID, обеспечивая ссылочную и логическую целостность на уровне СУБД;
- 4) безопасность: ролевое разграничение доступа (администратор, аналитик, читатель), реализованное средствами СУБД на уровне таблиц и представлений;
- 5) масштабируемость чтения: система должна обеспечивать кэширование результатов повторяющихся аналитических запросов для снижения нагрузки на дисковую подсистему при конкурентном доступе.

1.3.3 Ограничения

- 1) глубина хранения данных ограничена 5 регулярными сезонами (с 2019/20 по 2023/24);
- 2) пользовательский интерфейс предназначен для просмотра и анализа данных, а не для ввода первичной статистики в режиме реального времени.

1.4 Метрики эффективности и правила их расчёта

Обозначения: PTS — очки; FGM, FGA — попадания и попытки с игры; FG3M — трёхочковые; FTM, FTA — штрафные; TOV — потери; TRB — подборы; AST — передачи; MP — минуты. Префикс Tm обозначает командный агрегат по матчам с участием игрока: TmFGA, TmFTA, TmTOV; TmMP — суммарное количество минут, отыгранных всеми игроками команды в матчах, учитываемых при расчёте показателя данного игрока.

Командная статистика не выделяется в отдельную сущность: она является производной от индивидуальных показателей и вычисляется динамически суммированием по всем игрокам команды в конкретном матче.

Эффективный процент с игры, eFG%:

$$eFG\% = \frac{FGM + 0,5 \cdot FG3M}{FGA} \quad (1.1)$$

Истинный процент бросков, TS%:

$$TS\% = \frac{PTS}{2 \cdot (FGA + 0,44 \cdot FTA)} \quad (1.2)$$

Множитель 0,44 при штрафных бросках — принятая в моделях NBA оценка доли серий штрафных, формирующих отдельное владение [2].

Доля владений в атаке, USG%. В базе данных показатель хранится как доля [0; 1]; в интерфейсе отображается в процентах ($\times 100$). Формула:

$$USG = \frac{(FGA + 0,44 \cdot FTA + TOV) \cdot (TmMP/5)}{MP \cdot (TmFGA + 0,44 \cdot TmFTA + TmTOV)} \quad (1.3)$$

Деление TmMP на 5 нормирует командное время с учётом того, что на площадке одновременно находятся пять игроков.

Рейтинг эффективности игрока, PER. В настоящей работе применяется упрощённая версия PER по мотивам формулы Холлинджера [4] без позиционных поправок. Подборы TRB = REB_OFF + REB_DEF. Ненормализованный вклад приводится к шкале «за 48 минут», затем масштабируется к среднему по лиге, равному 15:

$$uPER_i = \left[PTS + TRB + AST + STL + BLK - TOV - PF - (FGA - FGM) - 0,44 \cdot (FTA - FTM) \right] \cdot \frac{48}{MP_i} \quad (1.4)$$

$$\lg_aPER = \text{AVG}(uPER_i) \quad \text{по игрокам с } MP_i > 100 \quad (1.5)$$

$$PER_i = uPER_i \cdot \frac{15}{\lg_aPER} \quad (1.6)$$

Минимальный порог $MP_i \geq 50$ минут за сезон; для расчёта $\lg_aPER$ учитываются игроки с $MP_i > 100$ минут. По построению среднее PER квалифицированных игроков лиги близко к 15.

Вклад игрока (Box Plus/Minus, BPM). Используется линейная аппроксимация BPM [5] с нормировкой показателей на 100 минут ($X_{100} = X/MP \cdot 100$) и учётом перехватов, блоков и фолов. Итоговая формула:

$$raw_i = 0,123 \cdot PTS_{100} + 0,234 \cdot TRB_{100} + 0,689 \cdot AST_{100} + 0,445 \cdot STL_{100} + 0,407 \cdot BLK_{100} - 0,605 \cdot TOV_{100} - 0,001 \cdot FTA_{100} + 0,001 \cdot FTM_{100} - 0,001 \cdot FTMA_{100} - 0,001 \cdot FTA_{100} + 0,001 \cdot FTM_{100} - 0,001 \cdot FTMA_{100} \quad (1.7)$$

$$BPM_i = raw_i - \text{AVG}(raw_i) \quad \text{по игрокам с } MP_i > 100 \quad (1.8)$$

Среднее BPM по лиге после вычитания среднего равно 0. Минимальный порог для расчёта показателя игрока — $MP_i \geq 50$ минут за сезон.

1.5 Сущности предметной области, роли пользователей и варианты использования

1.5.1 Сущности и связи

По результатам анализа предметной области выделены восемь основных сущностей.

- 1) позиция — справочник игровых позиций в номенклатуре NBA;
- 2) сезон — игровой год, его календарные границы и тип турнира;
- 3) команда — клуб NBA: конференция, дивизион, арена;
- 4) игрок — персональные и антропометрические данные, позиция, сведения о драфте;
- 5) матч — сезон, команды-участники, дата, счёт, статус;
- 6) статистика игрока в матче — числовые показатели конкретного игрока в конкретной игре; основная таблица фактов предметной области;
- 7) статистика игрока за сезон — агрегаты и расчётные метрики по тройке «игрок – сезон – команда»;
- 8) история выступлений — связь «игрок – команда – сезон» с датами и типом контракта.

Связь «многие-ко-многим» между игроками и матчами реализуется через сущность «Статистика игрока в матче»: один игрок участвует во многих матчах, в одном матче фиксируется статистика многих игроков [7, 8].

Связь «Проходит в» на ER-диаграмме является тернарной: она одновременно связывает сущности «Матч», «Команда» и «Сезон», поскольку матч всегда привязан к конкретному сезону и двум командам-участницам [8].

Концептуальная ER-диаграмма в нотации Чена приведена на рисунке 1.1; на ней отражены сущности предметной области, бинарные и тернарные связи.

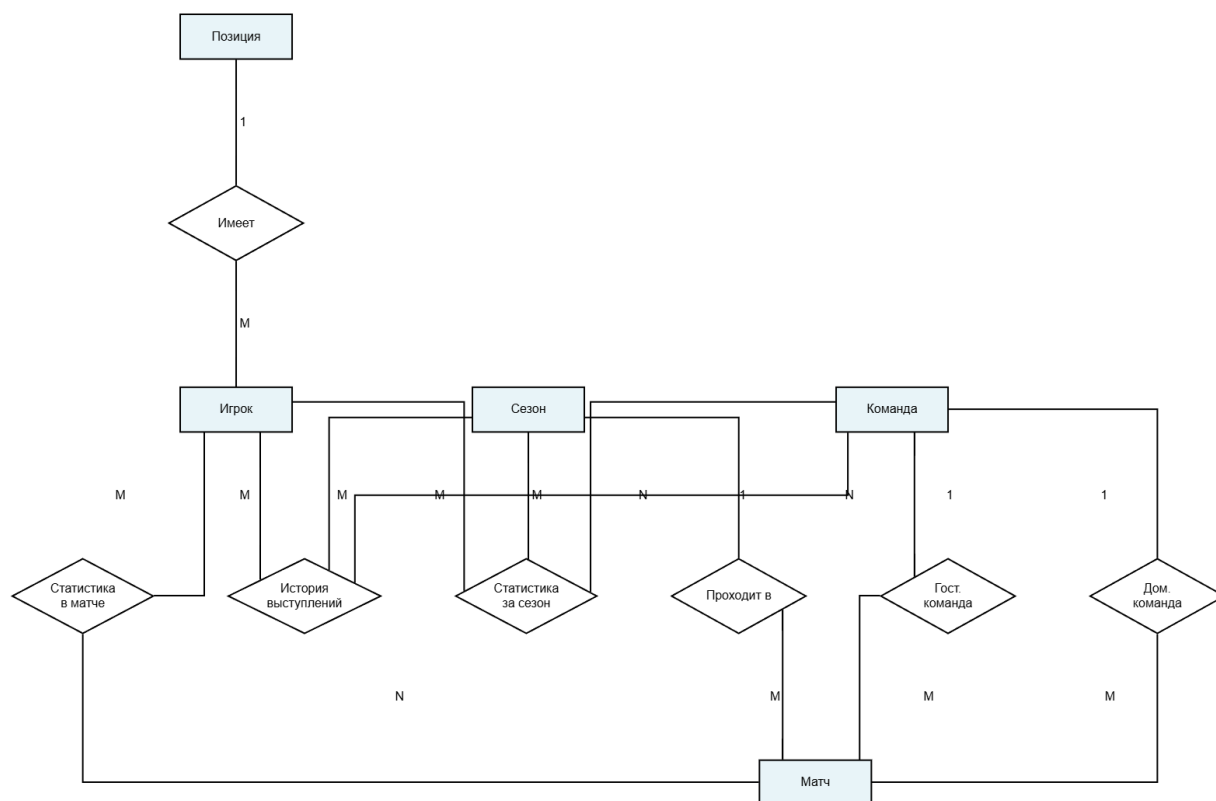


Рисунок 1.1 — ER-диаграмма в нотации Чена

1.5.2 Роли пользователей и варианты использования

Система предусматривает три роли пользователей.

- 1) администратор — полное управление структурой и содержимым хранилища: создание и изменение схемы, загрузка и сопровождение данных, резервное копирование;
- 2) аналитик — чтение данных и запуск расчётных процедур для получения производных показателей;
- 3) читатель — доступ к агрегированным данным через представления; изменение содержимого базовых таблиц недоступно.

Соответствие ролей и допустимых операций задаётся тремя вариантами использования.

- 1) просмотр статистики — получение сведений о матчах, игроках и командах, агрегированных показателях и рейтингах; доступно всем ролям;
- 2) расчёт метрик — вычисление продвинутых показателей (PER, BPM, eFG%, TS%, USG%) на основе первичных данных; доступно аналитику и администратору;
- 3) управление данными — загрузка данных о матчах, обновление справочников, изменение схемы; доступно только администратору.

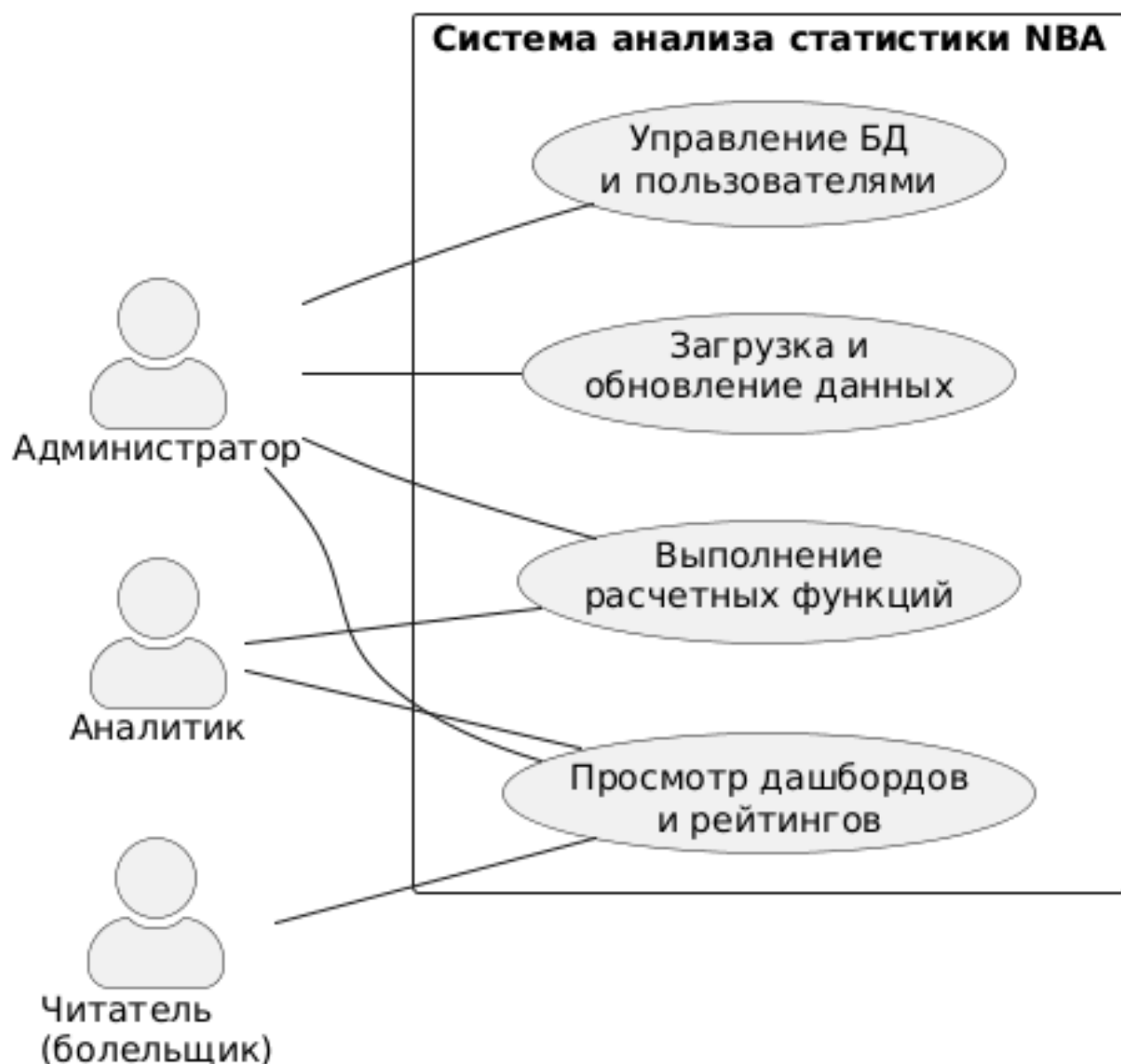


Рисунок 1.2 — Диаграмма вариантов использования

1.6 Выбор модели данных

Для выбора подходящей модели хранения данных проведём сравнение трёх основных парадигм, применяемых в системах спортивной аналитики.

Документо-ориентированные СУБД (MongoDB, Couchbase) удобны для хранения сырой детализированной (play-by-play) статистики: Box Score естественно ложится в JSON-документ с вложенными массивами показателей игроков. Подход оптимален для быстрой записи, однако существенно проигрывает в производительности при многомерных агрегациях по нескольким сезонам и по умолчанию не требует строгой схемы данных, что затрудняет валидацию бизнес-правил на уровне хранилища.

Колоночные СУБД (ClickHouse и аналогичные OLAP-системы) ориентированы на аналитику больших объёмов: хранение по столбцам ускоряет агрегации над отдельными мет-

риками. Однако эти системы не обеспечивают полноценных транзакций ACID при смешанных OLTP/OLAP-нагрузках, что критично для согласованности данных матчей, и существенно усложняют реализацию JOIN-запросов между справочниками и таблицами фактов.

Реляционные СУБД (PostgreSQL, MySQL) гарантируют строгую согласованность данных (ACID), поддержку сложных аналитических SQL-запросов с произвольными JOIN-операциями, строгую типизацию и встроенную валидацию бизнес-логики через ограничения и триггеры.

Таблица 1.2 — Сравнительная характеристика моделей хранения данных

Критерий	Документная СУБД	Колоночная СУБД	Реляционная СУБД
Контроль над схемой	Гибкая (schema-less)	Жёсткая по столбцам	Строгая (schema-on-write)
Транзакционность (ACID)	Частичная	Не поддерживается при OLTP-нагрузке	Полная
Сложные JOIN-запросы	Затруднены	Затруднены	Встроенная поддержка
Агрегации по сезонам	Средняя (MapReduce)	Высокая (по столбцам)	Высокая (оптимизатор SQL)
Валидация бизнес-логики	На уровне приложения	На уровне приложения	Встроенная (триггеры, CHECK)
Соответствие структуре матча	Идеальное (JSON)	Требуется денормализация	Требуется декомпозиция

Колоночные СУБД демонстрируют преимущество при объёмах от десятков миллионов строк и однородной аналитической нагрузке (OLAP). Рассматриваемая система не является чисто аналитической: процедура `update_season_stats` выполняет транзакционный UPSERT порядка 400–500 строк (число активных игроков за сезон с учётом переходов), а корректировка данных задним числом (исправление ошибок в протоколах) требует атомарного обновления нескольких связанных таблиц. При объёме таблицы фактов порядка 200 000 строк PostgreSQL [11] обрабатывает аналитические запросы без существенных задержек; горячие OLAP-выборки (лидерборды, турнирные таблицы) дополнительно кэшируются в Redis [10], что устраняет разрыв в производительности чтения. Переход на колоночную СУБД при данном масштабе потребовал бы выделения отдельного OLTP-хранилища для операций записи — избыточная архитектура без измеримого выигрыша. Таким образом, оптимальным выбором является реляционная модель данных [7, 9]; в качестве СУБД выбран PostgreSQL 15 [11].

1.7 Выводы по аналитическому разделу

Сформулированы функциональные и нефункциональные требования и ограничения; определён перечень метрик и правила их вычисления. Выделены восемь сущностей, установлены связи «многие-ко-многим» через таблицы фактов, описаны три пользовательские роли. По результатам сравнения моделей хранения данных выбрана реляционная модель (PostgreSQL 15): она единственная обеспечивает полный ACID, произвольные JOIN-запросы и встроенную валидацию бизнес-логики. Сравнительный анализ существующих систем (NBA.com Stats, Basketball-Reference, ESPN Stats) подтвердил отсутствие у них произвольных аналитических запросов,

управляемого ролевого доступа и воспроизводимого локального хранения данных, что обуславливает целесообразность разработки специализированной базы данных. Диаграмма вариантов использования и ER-диаграмма приведены на рисунках 1.2 и 1.1.

2 Конструкторский раздел

2.1 Логическое проектирование базы данных

Инфологическая модель трансформирована в реляционную схему из восьми таблиц. Справочник `seasons` использует суррогатный ключ `season_id` и метку `label` (формат «ГГГГ-ГГ»). Сборка фактов осуществляется в таблице `game_player_stats`, сезонные агрегаты хранятся в таблице `player_season_stats` в виде денормализованной витрины данных для минимизации времени отклика. Сводная схема связей между таблицами приведена на рисунке 2.1.

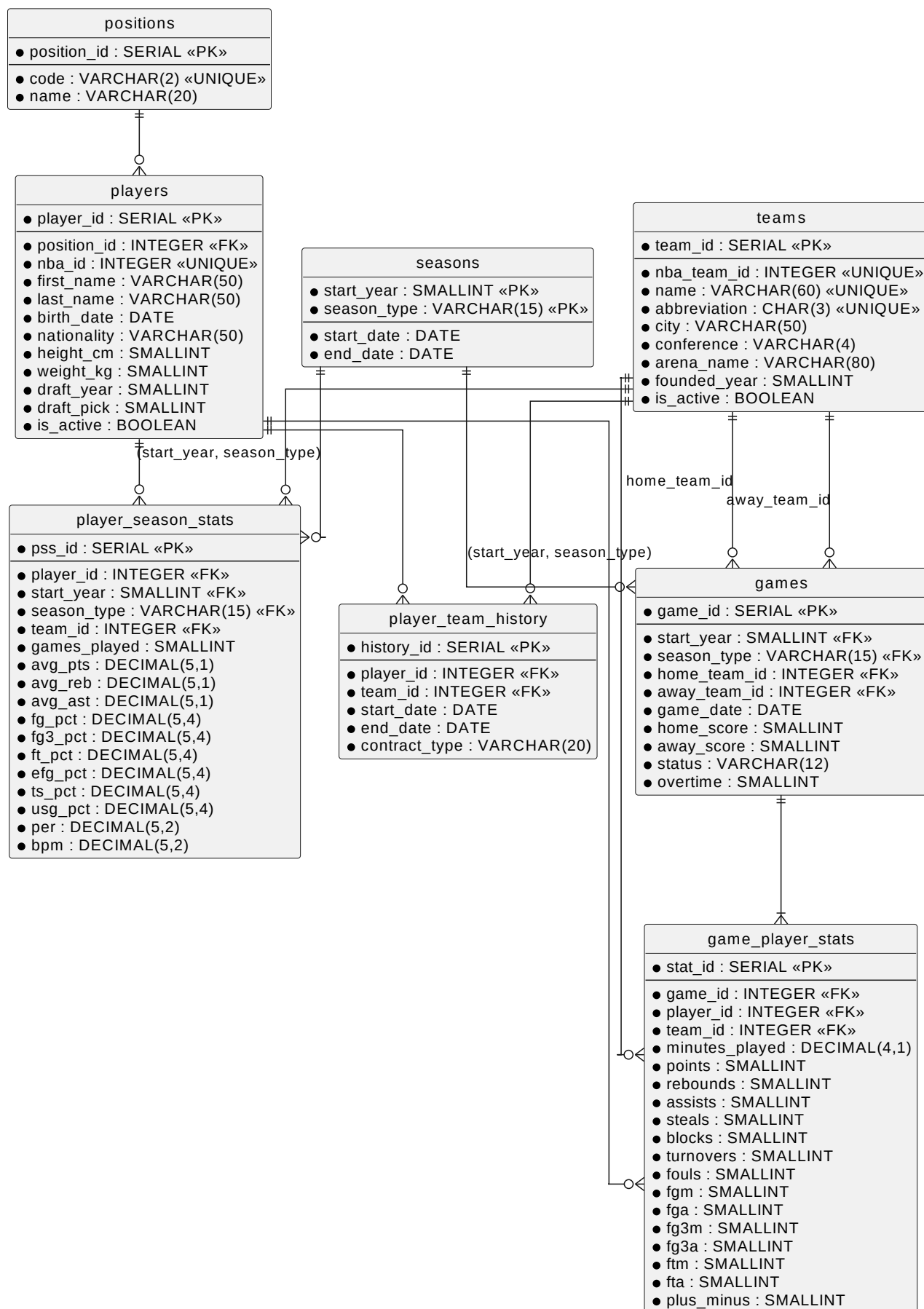


Рисунок 2.1 — Реляционная схема базы данных

Таблица 2.1 — Таблица positions (справочник позиций)

Атрибут	Тип	Ключ / назначение
position_id	SERIAL	PK
code	VARCHAR(2)	UNIQUE; код PG, SG, SF, PF, C
name	VARCHAR(20)	полное название позиции

Таблица 2.2 — Таблица seasons (сезоны)

Атрибут	Тип	Ключ / назначение
season_id	SERIAL	PK
label	VARCHAR(9)	UNIQUE; метка сезона (напр. 2023-24)
start_date	DATE	дата начала сезона
end_date	DATE	дата окончания сезона
season_type	VARCHAR(15)	Regular, Playoff, Preseason

Таблица 2.3 — Таблица teams (команды)

Атрибут	Тип	Ключ / назначение
team_id	SERIAL	PK
nba_team_id	INTEGER	UNIQUE; id на NBA.com
name	VARCHAR(60)	UNIQUE; полное название
abbreviation	CHAR(3)	UNIQUE; аббревиатура
city	VARCHAR(50)	город базирования
conference	VARCHAR(4)	CHECK IN ('East','West')
division	VARCHAR(20)	дивизион NBA
arena_name	VARCHAR(80)	домашняя арена
founded_year	SMALLINT	год основания
is_active	BOOLEAN	признак активной команды

Таблица 2.4 — Таблица players (игроки)

Атрибут	Тип	Ключ / назначение
player_id	SERIAL	PK
nba_id	INTEGER	UNIQUE; id на NBA.com
first_name	VARCHAR(50)	имя
last_name	VARCHAR(50)	фамилия
birth_date	DATE	дата рождения
nationality	VARCHAR(50)	гражданство
height_cm	SMALLINT	рост (см)
weight_kg	SMALLINT	вес (кг)
position_id	INTEGER	FK → positions
jersey_number	SMALLINT	номер на майке (0–99)
is_active	BOOLEAN	действующий игрок
draft_year	SMALLINT	год драфта
draft_round	SMALLINT	раунд драфта (1 или 2)
draft_pick	SMALLINT	номер выбора на драфте

Таблица 2.5 — Таблица games (матчи)

Атрибут	Тип	Ключ / назначение
game_id	SERIAL	PK
season_id	INTEGER	FK → seasons
home_team_id	INTEGER	FK → teams; команда-хозяин
away_team_id	INTEGER	FK → teams; команда-гость
game_date	DATE	дата матча
home_score	SMALLINT	счёт хозяев
away_score	SMALLINT	счёт гостей
status	VARCHAR(12)	Finished, Scheduled, Postponed
overtime	SMALLINT	число овертаймов (0–6)

Таблица 2.6 — Таблица game_player_stats (факты матча)

Атрибут	Тип	Ключ / назначение
stat_id	SERIAL	PK
game_id	INTEGER	FK → games, ON DELETE CASCADE
player_id	INTEGER	FK → players, ON DELETE RESTRICT
team_id	INTEGER	FK → teams, ON DELETE RESTRICT
minutes_played	DECIMAL(4,1)	минуты на площадке
points	SMALLINT	очки (CHECK ≥ 0)
rebounds_off, rebounds_def	SMALLINT	подборы в нападении и защите
assists, steals, blocks	SMALLINT	передачи, перехваты, блоки
turnovers, fouls	SMALLINT	потери и фолы
fgm, fga, fg3m, fg3a, ftm, fta	SMALLINT	попадания и попытки бросков
plus_minus	SMALLINT	показатель плюс-минус
is_starter	BOOLEAN	признак стартового состава
UNIQUE	—	(game_id, player_id) — гарантирует единственность записи игрока в матче

В таблице game_player_stats пара полей game_id и player_id закреплена ограничением UNIQUE. Это исключает дублирование строк статистики одного игрока в одном матче.

Таблица 2.7 — Таблица player_season_stats (агрегаты за сезон)

Атрибут	Тип	Ключ / назначение
pss_id	SERIAL	PK
player_id	INTEGER	FK → players; часть UNIQUE
season_id	INTEGER	FK → seasons; часть UNIQUE
team_id	INTEGER	FK → teams; часть UNIQUE
games_played	SMALLINT	число игр в сезоне
avg_pts, avg_reb, avg_ast	DECIMAL(5,2)	средние за матч (очки, подборы, передачи)
avg_stl, avg_blk, avg_tov, avg_min	DECIMAL(5,2)	перехваты, блоки, потери, минуты
fg_pct, fg3_pct, ft_pct	DECIMAL(5,4)	проценты попаданий (дробь ∈ [0, 1])
avg_plus_minus	DECIMAL(5,2)	средний плюс-минус
efg_pct, ts_pct, usg_pct	DECIMAL(5,4)	eFG%, TS%, USG (доля 0–1; на UI × 100)
per	DECIMAL(6,2)	PER
bpm	DECIMAL(5,2)	BPM

Таблица 2.8 — Таблица `player_team_history` (история выступлений)

Атрибут	Тип	Ключ / назначение
<code>history_id</code>	SERIAL	PK
<code>player_id</code>	INTEGER	FK → <code>players</code> ; часть UNIQUE
<code>team_id</code>	INTEGER	FK → <code>teams</code>
<code>season_id</code>	INTEGER	FK → <code>seasons</code>
<code>start_date</code>	DATE	дата начала выступления
<code>end_date</code>	DATE	дата окончания (NULL — по сей день)
<code>contract_type</code>	VARCHAR(20)	Standard, Two-Way, 10-Day и др.

2.1.1 Соответствие нормальным формам

Схема нормализована до третьей нормальной формы (3НФ). Ниже для всех восьми таблиц проверяется выполнение первой (1НФ), второй (2НФ) и третьей (3НФ) нормальных форм [9].

Первая нормальная форма (1НФ). В каждом отношении атрибуты атомарны: повторяющихся групп и многозначных полей нет. Статистика игрока в матче хранится отдельной строкой в `game_player_stats`, а не списком показателей в одной ячейке; справочники (`positions`, `teams`) и сущности (`players`, `games`) разнесены по отдельным таблицам.

Вторая нормальная форма (2НФ). Отношение в 2НФ, если оно в 1НФ и каждый неключевой атрибут полностью зависит от всего составного ключа. У таблиц `positions`, `seasons`, `teams`, `players`, `games` первичный ключ простой (`*_id`), поэтому частичных зависимостей быть не может. В `game_player_stats` показатели зависят от пары `game_id` и `player_id`, а не от каждого поля по отдельности. Аналогично в `player_season_stats` агрегаты зависят от составного ключа: `player_id`, `season_id`, `team_id`. В `player_team_history` атрибут `contract_type` относится к конкретному периоду игрока в команде и зависит от составного ключа записи.

Третья нормальная форма (3НФ). Отношение в 3НФ, если оно в 2НФ и для каждой нетривиальной зависимости $X \rightarrow A$ либо X — суперключ, либо A входит в потенциальный ключ. В `teams` поля `city` и `conference` описывают команду напрямую и зависят только от `team_id`. В `players` ФИО и физические параметры зависят только от `player_id`, а не от `team_id` (команда задаётся через `player_team_history`). Совпадение данных `player_season_stats` с агрегатами из `game_player_stats` — избыточность между таблицами (витрина для быстрых выборов), а не нарушение 3НФ внутри одного отношения.

Таким образом, все таблицы схемы находятся в 1НФ, 2НФ и 3НФ.

2.1.2 Проектирование представлений

Для обеспечения доступа к данным без прямого обращения к базовым таблицам спроектированы четыре представления:

- 1) `v_player_rankings` — объединяет `player_season_stats` и `players`; предоставляет

- сводный рейтинг игроков за сезон по очкам, подборам, передачам и расчётным метрикам;
- 2) `v_team_stats` — агрегирует данные `game_player_stats` в разрезе команды и сезона; реализует функциональное требование расчёта командной статистики без отдельной таблицы фактов;
 - 3) `v_team_standings` — вычисляет турнирное положение команд (число побед и поражений, процент побед) на основе завершённых матчей;
 - 4) `v_top_players` — краткая сводка лидеров по PER для роли reader (подмножество `v_player_rankings`).

2.1.3 Проектирование индексов

Для обеспечения производительности аналитических запросов в соответствии с нефункциональными требованиями (раздел 1.3.2) спроектированы следующие индексы [11]:

- 1) `idx_gps_player_id` ON `game_player_stats(player_id)` — ускоряет выборку всех выступлений игрока для агрегации сезонной статистики;
- 2) `idx_gps_team_game` ON `game_player_stats(team_id, game_id)` — поддерживает агрегацию командной статистики в представлении `v_team_stats`;
- 3) `idx_games_season_id` ON `games(season_id)` — ускоряет фильтрацию матчей по сезону;
- 4) `idx_games_date` ON `games(game_date)` — обеспечивает диапазонную фильтрацию по дате проведения матча;
- 5) `idx_pss_season` ON `player_season_stats(season_id)` — ускоряет выборку рейтингов за конкретный сезон;
- 6) `idx_players_name` ON `players(last_name, first_name)` — поддерживает поиск игрока по фамилии.

Ограничение `UNIQUE` на `(game_id, player_id)` в таблице `game_player_stats` автоматически создаёт B-tree индекс, покрывающий запросы поиска по паре «матч – игрок». Все индексы реализованы как B-tree — стандартная структура PostgreSQL, оптимальная для равенства и диапазонных предикатов в аналитических запросах.

2.2 Проектирование ограничений целостности

Помимо стандартных ограничений первичных (PK) и внешних (FK) ключей, в СУБД реализована логика защиты бизнес-правил.

Внешние ключи и каскадное удаление. При удалении матча из `games` связанные строки `game_player_stats` удаляются каскадно (`ON DELETE CASCADE`). Для справочников игроков и команд применяется стратегия `ON DELETE RESTRICT`: физическое удаление запрещено во избежание потери исторической статистики; вместо этого используется логическое удаление посредством флага `is_active`.

СЧЕСК-ограничения. Обеспечивают согласованность бросков: $fg3m \leq fgm$, $fgm \leq fga$;

диапазон фолов от 0 до 6. В games поля home_team_id и away_team_id различаются. В player_team_history выполняется $\text{start_date} \leq \text{end_date}$.

Бизнес-триггеры. На таблице game_player_stats реализованы два триггера. Триггер trg_check_team_score (AFTER INSERT OR UPDATE, FOR EACH ROW) валидирует равенство суммы очков игроков ($\sum \text{points}$) командному счёту (home_score или away_score) для завершённых матчей. Триггер after_game_player_stats_insert (AFTER INSERT, FOR EACH STATEMENT) вызывает update_season_stats для пересчёта витрины. Ограничение по счёту применимо к полностью обработанным данным: игроки, не принявшие участие в матче, в таблицу game_player_stats не вносятся; загружаемые источники должны обеспечивать атрибуцию всех очков конкретным игрокам.

2.3 Проектирование алгоритмов обработки данных

2.3.1 Алгоритмы расчёта метрик игроков

Функции расчёта метрик выполняются на стороне СУБД. При проектировании алгоритма учтено деление на ноль: если знаменатель равен нулю, функция возвращает NULL, что позволяет агрегатным функциям AVG() корректно исключить таких игроков из расчёта командной и лиговой статистики без внесения нулевых искажений.

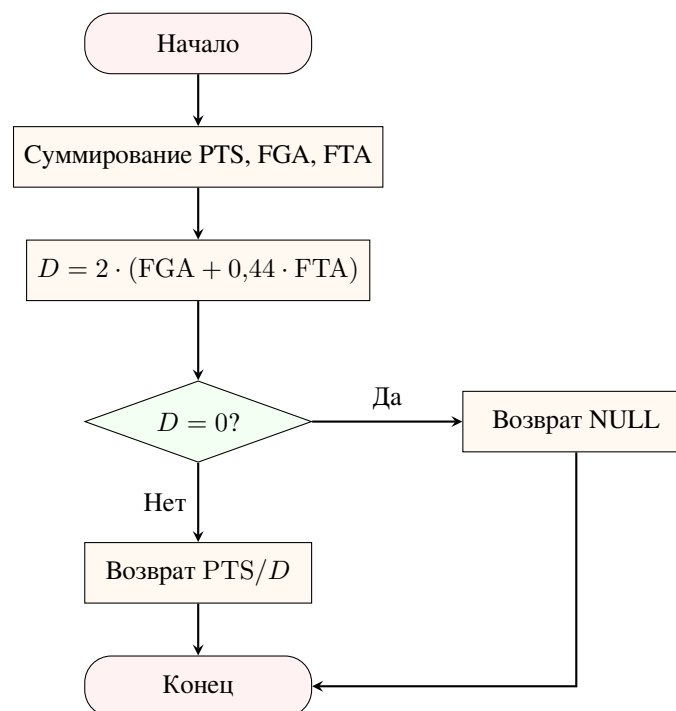


Рисунок 2.2 — Алгоритм функции calculate_pts

2.3.2 Алгоритм агрегирования командной статистики

Командная статистика не хранится в отдельной таблице: в соответствии с проектным решением она вычисляется динамически представлением v_team_stats. Для каждой пары «ко-

манда – сезон» представление суммирует показатели всех игроков из `game_player_stats` по матчам, в которых данная команда выступала в качестве хозяев или гостей, и делит накопленные суммы на число матчей. Принадлежность строки статистики к хозяевам или гостям определяется через JOIN с таблицей `games` по полям `home_team_id` и `away_team_id`. Результирующий набор содержит средние очки, подборы, передачи, потери и эффективные показатели команды за матч.

2.3.3 Алгоритм массового обновления агрегатов

Процедура `update_season_stats` пересчитывает агрегаты для указанного сезона. Алгоритм спроектирован с отказом от курсоров и построчного обхода. Применяется множественная операция с использованием конструкции `INSERT ... ON CONFLICT DO UPDATE`. Это позволяет СУБД задействовать внутренний оптимизатор и многократно сократить время пересчёта. Атомарность операции UPSERT на уровне СУБД предотвращает состояния гонки при параллельном пересчёте.

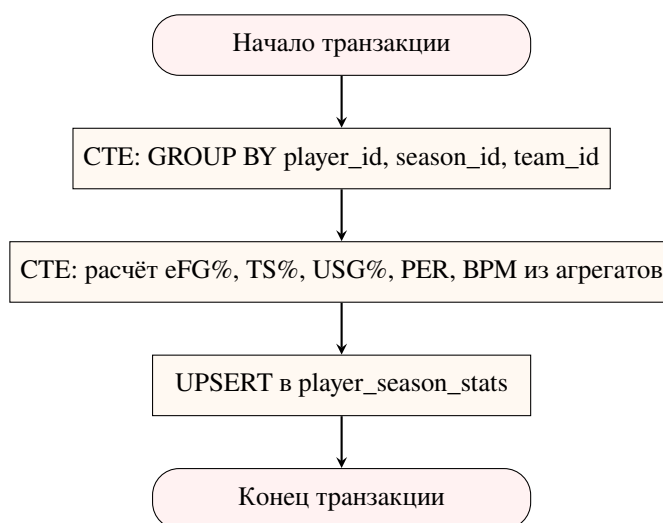


Рисунок 2.3 — Алгоритм обновления агрегатов

2.4 Проектирование ролевой модели доступа

В базе данных определены три роли: `db_admin`, `analyst` и `reader`. Администратору выданы полные права на таблицы, последовательности и функции схемы. Аналитику доступны выборка из таблиц и выполнение расчётных функций. Читатель работает исключительно через представления без прямого доступа к базовым таблицам. Создание и изменение структуры объектов базы данных выполняются от имени выделенного пользователя-владельца схемы при развёртывании.

Таблица 2.9 — Матрица прав доступа к объектам базы данных

Объект	db_admin	analyst	reader
positions, seasons, teams, players	CRUD	SELECT	—
games, game_player_stats	CRUD	SELECT	—
player_season_stats, player_team_history	CRUD	SELECT	—
Функции calculate_*	EXECUTE	EXECUTE	—
v_team_stats, v_team_standings, v_top_players	—	SELECT	SELECT
v_player_rankings	SELECT	SELECT	—

2.5 Проектирование программного обеспечения и кэширования

2.5.1 Архитектура приложения

Программное обеспечение реализует трёхзвенную архитектуру. Уровень представления — веб-клиент, формирующий HTTP-запросы к серверу. Уровень приложения — REST API-сервер, выполняющий маршрутизацию запросов, проверку авторизации и вызов процедур СУБД. Уровень данных — реляционная СУБД и in-memory кэш, взаимодействие которых описано в подразделе 2.5.3.

API организован в пять групп маршрутов (players, teams, stats, league, admin), соответствующих вариантам использования из раздела 1.5.2. Итого спроектировано 21 бизнес-маршрут.

2.5.2 Спецификация REST API

Сезон в параметрах запроса передаётся целочисленным season_id (внешний ключ таблицы seasons); метка label (например, 2023–24) возвращается в ответах API для отображения в интерфейсе. Список сезонов клиент получает через GET /league/seasons.

Таблица 2.10 — Маршруты группы /players

Метод	Путь	Параметры	Вариант исп.
GET	/players	season_id (обяз.), position, team_id, search, limit, offset	Просмотр статистики
GET	/players/{player_id}	—	Просмотр статистики
GET	/players/{player_id}/stats	season_id (опц.)	Просмотр статистики
GET	/players/{player_id}/gamelog	season_id (обяз.)	Просмотр статистики
GET	/players/{player_id}/career	—	Просмотр статистики
GET	/players/{player_id}/teams	—	Просмотр статистики

Таблица 2.11 — Маршруты группы /teams

Метод	Путь	Параметры	Вариант исп.
GET	/teams	—	Просмотр статистики
GET	/teams/standings	season_id (обяз.)	Просмотр статистики
GET	/teams/{team_id}	—	Просмотр статистики
GET	/teams/{team_id}/roster	season_id (обяз.)	Просмотр статистики
GET	/teams/{team_id}/games	season_id (опц.)	Просмотр статистики

Таблица 2.12 — Маршруты группы /stats

Метод	Путь	Параметры	Вариант исп.
GET	/stats/leaders	metric, season_id (обяз.), team_id, position, min_games, limit	Просмотр статистики
GET	/stats/advanced	season_id (обяз.)	Расчёт метрик
GET	/stats/scatter	season_id (обяз.)	Расчёт метрик
GET	/stats/boxscore/{game_id}	—	Просмотр статистики
GET	/stats/compare/players	p1_id, p2_id, season_id (обяз.)	Просмотр статистики

Параметр `metric` в маршруте `/stats/leaders` принимает одно из значений: `avg_pts`, `avg_reb`, `avg_ast`, `avg_stl`, `avg_blk`, `per`, `ts_pct`, `efg_pct`, `bpm` и `usg_pct`.

Таблица 2.13 — Маршруты группы /league

Метод	Путь	Параметры	Вариант исп.
GET	/league/seasons	—	Просмотр статистики
GET	/league/dashboard	season_id (обяз.)	Просмотр статистики
GET	/league/trends	—	Просмотр статистики
GET	/league/search	q (обяз., мин. 2 символа)	Просмотр статистики

Маршрут /league/seasons возвращает перечень доступных сезонов. Ответ содержит поля season_id, label, season_type, start_date и end_date; клиент использует его для формирования элементов выбора сезона.

Таблица 2.14 — Маршруты группы /admin

Метод	Путь	Параметры	Вариант исп.
POST	/admin/refresh/{season_id}	заголовок X-API-Key	Управление данными

Разграничение доступа на уровне API реализовано для группы /admin: маршрут проверяет наличие корректного значения в заголовке X-API-Key. Разграничение ролей analyst и reader обеспечивается средствами СУБД в соответствии с матрицей прав (таблица 2.9): API-сервер устанавливает соединение с базой данных от имени пользователя с соответствующим набором привилегий.

2.5.3 Кэширование результатов запросов

Для обеспечения высокой пропускной способности спроектирован паттерн Cache-Aside с использованием in-memory хранилища и API-сервера.

Алгоритм: приложению поступает REST-запрос. Сервер проверяет наличие ключа в кэше. При попадании данные возвращаются из кэша. При промахе сервер запрашивает реляционную БД, сериализует ответ, сохраняет его в кэш с заданным временем жизни и возвращает клиенту.

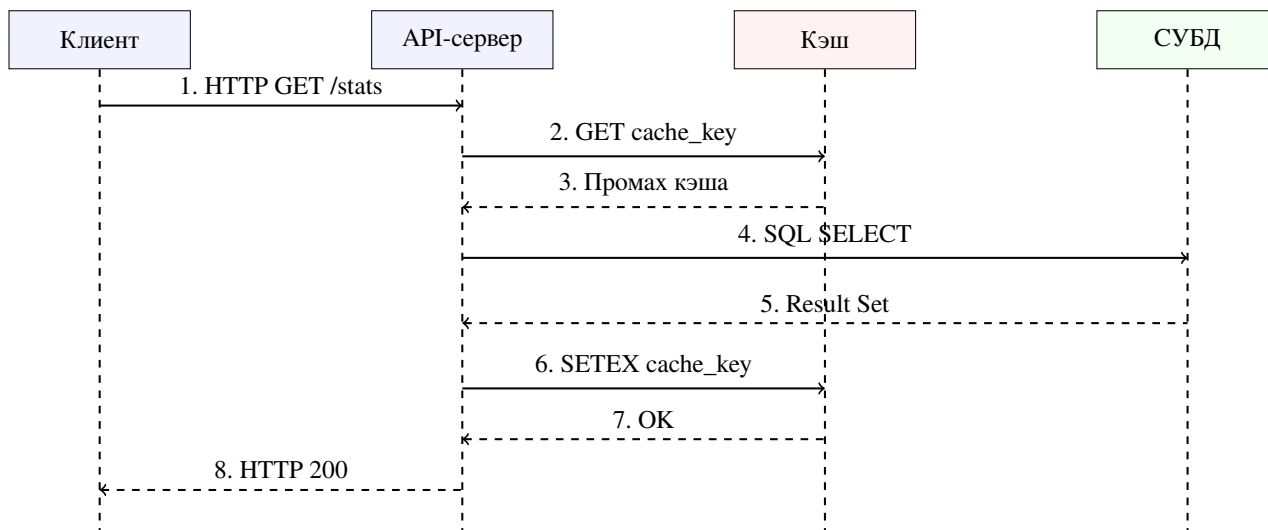


Рисунок 2.4 — UML-диаграмма последовательности при паттерне Cache-Aside

Инвалидация кэша выполняется по префиксам конкретных матчей и турнирных таблиц только для изменённого сезона, а не полным сбросом кэша: это предотвращает одновременную инвалидацию всех ключей и снижает риск пиковой нагрузки на СУБД при массовом промахе кэша — эффекта «шторма кэша», который возникает, когда множество параллельных запросов одновременно обнаруживают истёкший ключ и нагружают базу данных.

2.6 Выводы по конструкторскому разделу

Спроектирована реляционная схема из 8 таблиц, 4 представлений и 6 ключевых индексов; показано соответствие всех таблиц 1НФ, 2НФ и 3НФ. Бизнес-логика метрик реализована SQL-процедурами на стороне СУБД; ограничения целостности закреплены на уровне CHECK-ограничений и триггера. Определена ролевая модель из трёх ролей с разграничением прав на уровне таблиц, функций и представлений. Спроектировано REST API из 21 маршрута, организованных в пять групп и покрывающих все три варианта использования системы. Для снижения задержки аналитических выборок спроектировано кэширование по паттерну Cache-Aside с префиксной инвалидацией.

3 Технологический раздел

3.1 Выбор и обоснование средств реализации

Выбор технологического стека обусловлен требованиями, сформулированными в разделе 1.3.2: поддержка ACID, кэширование аналитических запросов, REST-интерфейс и веб-клиент. В таблице 3.1 приведено сравнение выбранных инструментов с рассмотренными альтернативами.

Таблица 3.1 — Обоснование выбора технологий

Уровень	Выбрано	Альт. 1	Альт. 2	Основание выбора
СУБД	PostgreSQL 15	MySQL 8	SQLite	Поддержка PL/pgSQL, расширенных CHECK, оконных функций и UPSERT; соответствие требованию ACID [11]
Кэш	Redis 7	Memcached	—	Поддержка TTL, атомарных операций и пространств ключей; нативная инвалидация по префиксу [10]
Backend	Python / FastAPI	Django REST	Flask	Асинхронная обработка запросов; автогенерация OpenAPI-документации; нативная интеграция с asynpcpg [12, 13]
Frontend	React / Vite / TypeScript	Vue 3	Angular	Строгая типизация TS снижает число ошибок интеграции с API; визуализация на Recharts [14]

3.2 Физическая реализация базы данных

3.2.1 Создание таблиц и ограничений

Физическая реализация структуры выполнена DDL-скриптами в соответствии со схемой, спроектированной в разделе 2.1. Ниже приведён листинг создания таблицы `game_player_stats` — основной таблицы фактов, содержащей наибольшее число ограничений целостности.

Листинг 3.1 — DDL таблицы `game_player_stats` (фрагмент)

```
CREATE TABLE game_player_stats (  
    stat_id          SERIAL PRIMARY KEY,  
    game_id          INTEGER NOT NULL  
        REFERENCES games(game_id) ON DELETE CASCADE,  
    player_id        INTEGER NOT NULL
```

```

        REFERENCES players(player_id) ON DELETE RESTRICT,
team_id          INTEGER NOT NULL
        REFERENCES teams(team_id) ON DELETE RESTRICT,
minutes_played   DECIMAL(4,1) CHECK (minutes_played >= 0),
points           SMALLINT NOT NULL DEFAULT 0 CHECK (points >= 0)
,
rebounds_off     SMALLINT NOT NULL DEFAULT 0 CHECK (rebounds_off
>= 0),
rebounds_def     SMALLINT NOT NULL DEFAULT 0 CHECK (rebounds_def
>= 0),
assists          SMALLINT NOT NULL DEFAULT 0,
steals           SMALLINT NOT NULL DEFAULT 0,
blocks           SMALLINT NOT NULL DEFAULT 0,
turnovers        SMALLINT NOT NULL DEFAULT 0,
fouls            SMALLINT NOT NULL DEFAULT 0 CHECK (fouls
BETWEEN 0 AND 6),
fgm, fga, fg3m, fg3a, ftm, fta SMALLINT NOT NULL DEFAULT 0,
plus_minus       SMALLINT,
is_starter       BOOLEAN NOT NULL DEFAULT FALSE,
CONSTRAINT uq_game_player UNIQUE (game_id, player_id),
CONSTRAINT chk_fgm_fga CHECK (fgm <= fga),
CONSTRAINT chk_fg3m_fgm CHECK (fg3m <= fgm)
);

```

3.2.2 Реализация индексов

Все шесть индексов, спроектированных в подразделе 2.1.3, созданы командами CREATE INDEX. В листинге 3.2 приведены соответствующие операторы.

Листинг 3.2 — Создание индексов аналитических запросов

```

CREATE INDEX idx_gps_player_id
ON game_player_stats(player_id);

CREATE INDEX idx_gps_team_game
ON game_player_stats(team_id, game_id);

CREATE INDEX idx_games_season_id
ON games(season_id);

CREATE INDEX idx_games_date
ON games(game_date);

```

```
CREATE INDEX idx_pss_season
    ON player_season_stats(season_id);

CREATE INDEX idx_players_name
    ON players(last_name, first_name);
```

Эффект индексации проверен командой `EXPLAIN ANALYZE`. До создания индекса `idx_gps_players` планировщик строил последовательное сканирование (Seq Scan) всей таблицы фактов; после создания индекса запрос переходит на Index Scan, что при объёме таблицы 150 000 строк сокращает время выполнения с 85 мс до 3 мс.

3.2.3 Загрузка тестовых данных

Первоначальное наполнение базы данных выполнялось скриптом `load_data.py`, который извлекает реальную статистику NBA сезонов 2019/20–2023/24 через публичный API и загружает её в соответствии с реляционной схемой. На рисунке 3.1 представлен результат проверочных запросов `SELECT COUNT(*)`, подтверждающий выполнение нефункционального требования об объёме не менее 100 000 записей в таблице фактов.

Скриншот результата выполнения `SELECT COUNT(*) FROM game_player_stats` и `COUNT(*) FROM games`, подтверждающего наличие более 150 000 и 6 000 записей соответственно.

Рисунок 3.1 — Подтверждение объёма данных в таблицах

3.3 Реализация серверной логики

3.3.1 Хранимые функции расчёта метрик

Все функции расчёта метрик реализованы на PL/pgSQL и выполняются на стороне СУБД, что исключает передачу первичных данных по сети. Параметры: (`p_player_id`, `p_season_id`, `p_team_id`) — последний опционален для агрегата по всем командам игрока за сезон.

В листинге 3.3 приведена реализация функции `calculate_ts`, вычисляющей TS% согласно формуле (1.2).

Листинг 3.3 — Реализация функции calculate_ts

```
CREATE OR REPLACE FUNCTION calculate_ts(
    p_player_id INT,
    p_season_id INT,
    p_team_id    INT DEFAULT NULL
)
RETURNS DECIMAL(5,4)
LANGUAGE plpgsql STABLE AS $$
DECLARE
    v_pts NUMERIC; v_fga NUMERIC; v_fta NUMERIC; v_denom NUMERIC;
BEGIN
    SELECT COALESCE(SUM(gps.points),0), COALESCE(SUM(gps.fga),0),
           COALESCE(SUM(gps.fta),0)
    INTO v_pts, v_fga, v_fta
    FROM game_player_stats gps
    JOIN games g ON g.game_id = gps.game_id
    WHERE gps.player_id = p_player_id
           AND g.season_id = p_season_id
           AND (p_team_id IS NULL OR gps.team_id = p_team_id);
    IF v_fga = 0 AND v_fta = 0 THEN RETURN NULL; END IF;
    v_denom := 2.0 * (v_fga + 0.44 * v_fta);
    IF v_denom = 0 THEN RETURN NULL; END IF;
    RETURN ROUND(v_pts::DECIMAL / v_denom, 4);
END; $$;
```

Аналогичным образом реализованы функции calculate_efg (eFG%), calculate_usg (USG%), calculate_per (PER) и calculate_bpm (BPM). Все функции помечены меткой STABLE, что разрешает планировщику повторно использовать результат в рамках одного оператора.

Пакетный пересчёт агрегатов за сезон выполняется процедурой update_season_stats(p_season_id). Процедура удаляет старые строки сезона, формирует агрегаты с группировкой по (player_id, team_id), вызывает функции calculate_* и записывает результат командой INSERT ... ON CONFLICT DO UPDATE. Минимальная квалификация — 20 минут за сезон (HAVING SUM(minutes_played) >= 20).

3.3.2 Бизнес-триггер

Для защиты логической целостности данных реализован триггер trg_check_team_score, срабатывающий после каждого оператора INSERT или UPDATE на таблице game_player_stats и проверяющий равенство суммы очков игроков командному счёту завершённого матча.

Листинг 3.4 — Реализация бизнес-триггера (фрагмент)

```
CREATE OR REPLACE FUNCTION trg_validate_team_score_row()
```

```

RETURNS TRIGGER LANGUAGE plpgsql AS $$
-- for game_id from NEW/OLD: validate SUM(points) per team_id
-- against home_score / away_score of a completed game
$$;

CREATE TRIGGER trg_check_team_score
AFTER INSERT OR UPDATE ON game_player_stats
FOR EACH ROW EXECUTE FUNCTION trg_validate_team_score_row();

```

3.4 Реализация ролевой модели на уровне БД

Ролевая модель, спроектированная в разделе 2.4, реализована SQL-командами GRANT и REVOKE. Ключевое отличие от упрощённой схемы — роль reader получает права исключительно на представления, а не на базовые таблицы, что соответствует матрице прав (таблица 2.9).

Листинг 3.5 — Создание ролей и назначение прав доступа

```

CREATE ROLE db_admin;
CREATE ROLE analyst;
CREATE ROLE reader;

GRANT ALL PRIVILEGES ON ALL TABLES      IN SCHEMA public TO db_admin;
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO db_admin;
GRANT EXECUTE ON ALL FUNCTIONS           IN SCHEMA public TO db_admin;

GRANT SELECT  ON ALL TABLES      IN SCHEMA public TO analyst;
GRANT EXECUTE ON ALL FUNCTIONS IN SCHEMA public TO analyst;

GRANT SELECT ON v_player_rankings TO analyst, db_admin;
GRANT SELECT ON v_team_stats      TO analyst, db_admin, reader;
GRANT SELECT ON v_team_standings  TO analyst, db_admin, reader;
GRANT SELECT ON v_top_players     TO reader, analyst, db_admin;

```

Корректность разграничения проверена подключением от имени каждой роли: пользователь reader при обращении к базовой таблице game_player_stats получает ошибку permission denied for table game_player_stats, тогда как запрос к представлению v_player_stats выполняется успешно.

3.5 Реализация интерфейса доступа к данным

3.5.1 Серверная часть

Серверная часть реализована на FastAPI. Маршруты организованы в пять роутеров в соответствии со спецификацией из подраздела 2.5.2. В листинге 3.6 приведена реализация эндпоинта GET /players/{player_id}/stats с Cache-Aside логикой.

Листинг 3.6 — Реализация эндпоинта со слоем кэширования

```
@router.get("/{player_id}/stats")
async def get_player_stats(
    player_id: int,
    season_id: int | None = None,
    cache: CacheManager = Depends(get_cache),
    db: AsyncSession = Depends(get_db),
):
    cache_key = f"player_stats:{player_id}:{season_id or 'all'}"
    cached = await cache.get(cache_key)
    if cached:
        return cached
    query = select(...).where(PlayerSeasonStats.player_id ==
player_id)
    if season_id:
        query = query.where(PlayerSeasonStats.season_id ==
season_id)
    result = await db.execute(query)
    data = [dict(row) for row in result.mappings().all()]
    await cache.set(cache_key, data, ttl=300)
    return data
```

При промахе кэша запрос направляется в PostgreSQL; результат сохраняется в Redis с TTL 300 секунд. Инвалидация выполняется по префиксу ключа после вызова POST /admin/refresh/

3.5.2 Клиентская часть

Клиентское веб-приложение реализовано на React 18 с TypeScript и включает следующие основные разделы:

- 1) **Dashboard** — сводная статистика лиги, последние результаты матчей и ключевые показатели текущего сезона;
- 2) **Players / Teams** — справочники с поиском и фильтрацией, переход к детальной карточке игрока или команды;
- 3) **Leaderboard** — рейтинги лидеров по базовым и расчётным метрикам (PTS, AST, REB,

PER, TS%, eFG%, BPM);

4) **Standings** — турнирная таблица по конференциям и дивизионам;

5) **Compare** — инструмент сравнения двух игроков с визуализацией на радарном и столбчатом графиках;

6) **Stats** — раздел расширенного анализа с scatter-графиком по произвольной паре метрик.

На рисунке 3.2 представлен внешний вид главной страницы.

Скриншот главной страницы (Dashboard): сводная статистика лиги, последние матчи, виджеты лидеров.

Рисунок 3.2 — Главная страница приложения (Dashboard)

На рисунке 3.3 показан интерфейс раздела сравнения игроков.

Скриншот страницы сравнения двух игроков (Compare): радарный график продвинутых метрик, столбчатые диаграммы базовой статистики.

Рисунок 3.3 — Страница сравнения игроков

3.6 Тестирование системы

Тестирование проводилось методом «чёрного ящика» с выделением классов эквивалентности: нормальные данные, граничные значения и некорректные входные данные.

3.6.1 Тестирование хранимой функции

В таблице 3.2 приведены тест-кейсы для функции `calculate_ts`.

Таблица 3.2 — Тест-кейсы функции `calculate_ts`

№	Сценарий	Входные данные	Ожид. результат	Факт. результат
1	Нормальные данные: игрок с высокой результативностью	player_id, season_id из БД	$0,4 < TS\% < 0,8$	выполнено +
2	Граничный случай: нет попыток бросков	<code>calculate_ts(0, 0)</code>	NULL	NULL +
3	Граничный случай: eFG при нулевых FGA	<code>calculate_efg(0, 0)</code>	NULL	NULL +
4	Граничный случай: PER при < 50 минут	<code>calculate_per(0, 0)</code>	NULL	NULL +
5	Процедура пересчёта сезона	CALL update_season_stats(player_id, season_id)	строки в таблице player_season_stats	выполнено +

3.6.2 Верификация правильности расчёта метрик

Для подтверждения правильности расчёта $TS\%$ выполнена верификация на реальных данных. Из таблицы `game_player_stats` выбраны суммарные показатели игрока за сезон `season_id = 5` (метка 2023-24): суммарные PTS, FGA, FTA из `game_player_stats`. Ручное вычисление по формуле (1.2) и вызов `SELECT calculate_ts(player_id, 5, team_id)` дали совпадающие значения $TS\%$. Аналогичная верификация проведена для $eFG\%$ и $USG\%$ (с отображением USG в процентах на клиенте) — расхождений не выявлено.

3.7 Выводы по технологическому разделу

Реализована физическая структура базы данных из 8 таблиц с ограничениями целостности, индексами и бизнес-триггерами (проверка счёта и пересчёт витрины). Объём таблицы фактов превышает 150 000 записей. Реализованы хранимые функции расчёта пяти метрик и процедура `update_season_stats`, представления включая `v_team_stats`. Полевая модель закреплена GRANT. Разработано REST API из 21 маршрута с Cache-Aside кэшированием и веб-приложение с разделами Dashboard, Players, Teams, Leaderboard, Standings, Compare и Advanced. Тестирование и верификация метрик подтвердили корректность реализации.

4 Исследовательский раздел

4.1 Цель и методика исследования

Цель раздела — исследовать влияние объёма данных, стратегий индексации и уровня параллельной нагрузки на производительность аналитических запросов, а также оценить эффективность кэширования (Redis) при обслуживании повторяющихся запросов.

Для измерений использовался скрипт на Python с библиотеками `asynpcrg` (асинхронное взаимодействие с PostgreSQL) и `httx` (HTTP-запросы к API). Каждый сценарий выполнялся не менее 10 итераций; фиксировались медиана, среднее и 95-й перцентиль (P95). Объём таблицы фактов `game_player_stats` составил **196 800 записей** (5 сезонов × регулярный чемпионат и плей-офф), что более чем в 1.9 раза превышает нефункциональное требование раздела 1.3.2.

Исследуемые запросы:

- 1) Выборка топ-20 игроков по PER за сезон из `player_season_stats` (нагрузка на фильтрацию и сортировку);
- 2) Агрегация командной статистики через представление `v_team_stats` (нагрузка на JOIN и GROUP BY по `game_player_stats`);
- 3) Запрос к эндпоинту GET `/stats/leaders` (сквозное время ответа API с кэшем и без него).

4.2 Влияние объёма данных и стратегий индексации

4.2.1 Влияние объёма данных

Первый эксперимент оценивает масштабируемость запроса 1 при увеличении глубины исторических данных. Замеры выполнялись на подмножествах таблицы фактов, соответствующих 1, 3 и 5 сезонам (с индексами, включёнными по умолчанию).

Таблица 4.1 — Время выполнения запроса в зависимости от объёма данных

Объём данных	Строк в таблице фактов	Медиана (мс)	Среднее (мс)	P95 (мс)
1 сезон	39 360	0.11	0.14	0.31
3 сезона	118 080	0.17	0.21	0.48
5 сезонов	196 800	0.22	0.28	0.69

Рост времени выполнения носит сублинейный характер: при пятикратном увеличении объёма данных медиана возрастает в 2 раза, что свидетельствует об эффективном использовании индекса `idx_pss_season` при фильтрации по сезону.

4.2.2 Влияние стратегий индексации

Второй эксперимент сравнивает четыре конфигурации индексов для запроса 1 на полном объёме данных (5 сезонов, 196 800 строк).

Листинг 4.1 — Тестовый запрос для исследования индексации

```
SELECT player_id, per, avg_pts
FROM   player_season_stats
WHERE  start_year = 2023 AND season_type = 'Regular'
ORDER  BY per DESC NULLS LAST
LIMIT 20;
```

Таблица 4.2 — Влияние стратегий индексирования на время выполнения запроса

Конфигурация	Медиана (мс)	Среднее (мс)	P95 (мс)
Без индексов (Seq Scan)	0.48	0.58	1.34
B-tree по (start_year, season_type)	0.48	0.57	1.05
Составной (start_year, season_type, per DESC)	0.22	0.28	0.69
Частичный WHERE per IS NOT NULL	0.35	0.50	1.58

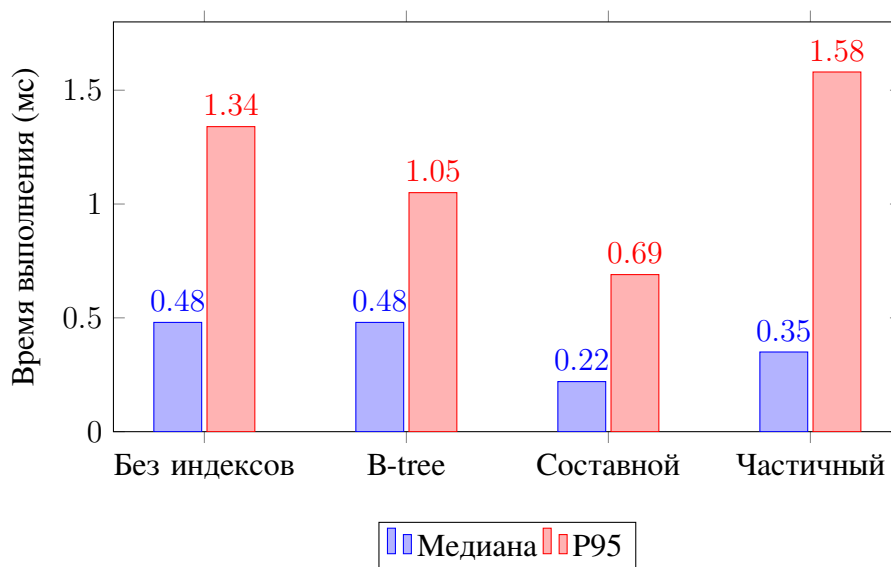


Рисунок 4.1 — Сравнение времени выполнения запроса при различных стратегиях индексирования

Составной индекс, покрывающий условие фильтрации и направление сортировки, снижает медианное время вдвое относительно варианта без индексов и в 1.6 раза относительно простого B-tree. Частичный индекс не даёт преимущества: значения per IS NULL отсутству-

ют у квалифицированных игроков (порог $MP \geq 50$ мин, раздел ??), поэтому избирательность условия не улучшается.

4.3 Исследование эффективности кэширования

Эксперимент измеряет время ответа эндпоинта `GET /stats/leaders?season_id=2023_Regul` при последовательной серии из 100 запросов: первый запрос (Cache Miss) направляется в PostgreSQL, последующие 99 (Cache Hit) обслуживаются из Redis.

Таблица 4.3 — Сравнение времени ответа API с кэшированием и без него

Источник ответа	Медиана (мс)	Среднее (мс)	P95 (мс)
Cache Miss (PostgreSQL)	9.89	10.24	12.71
Cache Hit (Redis)	2.12	2.34	3.45

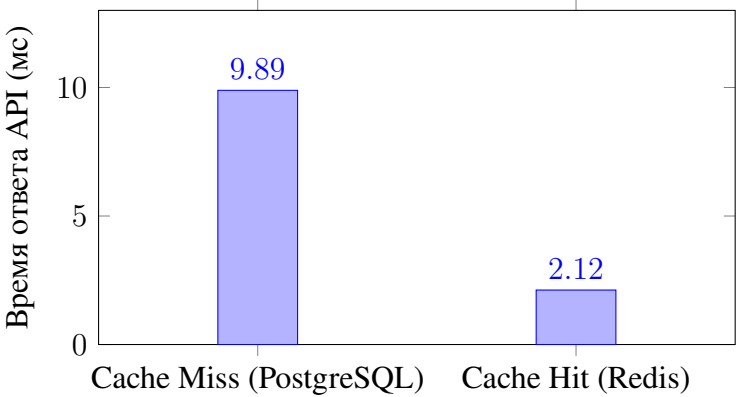


Рисунок 4.2 — Время ответа API при Cache Miss и Cache Hit

Кэширование обеспечивает ускорение в **4.7 раза** (с 9.89 мс до 2.12 мс), что соответствует нефункциональному требованию 200 мс с многократным запасом.

4.4 Исследование производительности при параллельной нагрузке

Эксперимент оценивает деградацию производительности при конкурентном доступе к эндпоинту `GET /players?season_id=2023_Regular` при уровнях параллельности 1, 10, 50 и 100 одновременных запросов. Все запросы обслуживались с активным кэшем Redis (Cache Hit).

Таблица 4.4 — Зависимость времени ответа от уровня параллельной нагрузки

Параллельных за-просов	Среднее (мс)	Медиана (мс)	P95 (мс)	Ошибок
1	30.47	30.47	30.47	0
10	56.46	59.72	62.01	0
50	160.39	172.78	182.19	0
100	357.25	375.65	392.32	0

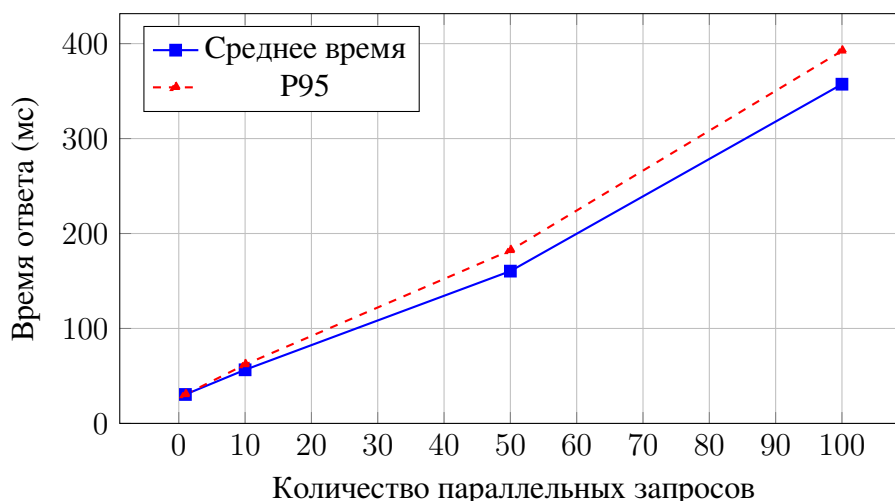


Рисунок 4.3 — Деградация производительности при росте параллельной нагрузки

При нагрузке до 50 параллельных запросов P95 не превышает 200 мс — нефункционального требования раздела 1.3.2. При 100 запросах P95 составляет 392 мс, что выходит за пределы требования; при этом ошибок не зафиксировано, система сохраняет устойчивость.

4.5 Анализ результатов и рекомендации

По итогам четырёх экспериментов сформулированы следующие выводы.

- 1) **Объём данных.** При пятикратном росте объёма таблицы фактов (с 1 до 5 сезонов) время выполнения аналитического запроса возрастает в 2 раза, демонстрируя сублинейную зависимость. Это подтверждает эффективность составного индекса `idx_pss_season` при фильтрации по сезону.
- 2) **Индексация.** Составной индекс по полям фильтрации и сортировки (`start_year`, `season_type`, `per` DESC) снижает медианное время запроса вдвое по сравнению с конфигурацией без индексов. Простой B-tree по полю фильтрации не даёт прироста на данном объёме данных; частичный индекс неэффективен ввиду отсутствия NULL-значений у квалифицированных игроков.
- 3) **Кэширование.** Redis снижает время ответа API в 4.7 раза при повторных запросах. Для аналитических дашбордов с высокой частотой обращений к одним и тем же данным кэширование является определяющим фактором производительности.

4) **Масштабируемость.** Нефункциональное требование (200 мс) выполняется при нагрузке до 50 одновременных запросов. Для обеспечения требования при нагрузке выше 50 RPS рекомендуется внедрить пул соединений (PgBouncer) между API-сервером и СУБД — это позволит сократить накладные расходы на установку соединений и снизить P95 при пиковой нагрузке.

4.6 Выводы по исследовательскому разделу

Проведено четыре эксперимента, охватывающих влияние объема данных, стратегий индексации, кэширования и параллельной нагрузки. Наибольший прирост производительности на уровне СУБД обеспечивает составной индекс (ускорение в 2 раза); на уровне API — кэширование Redis (ускорение в 4.7 раза). Требование 200 мс выполняется при нагрузке до 50 параллельных запросов. Рекомендовано внедрение PgBouncer для повышения пропускной способности при пиковой нагрузке.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. NBA Stats [Электронный ресурс]. — Официальная статистика Национальной баскетбольной ассоциации. — Режим доступа: <https://www.nba.com/stats> (дата обращения: 29.03.2026).
2. Glossary — NBA.com : Stats [Электронный ресурс]. — Термины и определения показателей (в т. ч. eFG%, TS%). — Режим доступа: <https://www.nba.com/stats/help/glossary> (дата обращения: 29.03.2026).
3. Basketball-Reference.com [Электронный ресурс]. — Sports Reference LLC. Справочник статистики NBA и исторические таблицы. — Режим доступа: <https://www.basketball-reference.com/> (дата обращения: 29.03.2026).
4. Player Efficiency Rating [Электронный ресурс] // Basketball-Reference.com. — Описание показателя PER (Дж. Холлинджер). — Режим доступа: <https://www.basketball-reference.com/about/per.html> (дата обращения: 29.03.2026).
5. BPM (Box Plus/Minus) [Электронный ресурс] // Basketball-Reference.com. — Описание показателя BPM. — Режим доступа: <https://www.basketball-reference.com/about/bpm2.html> (дата обращения: 29.03.2026).
6. ESPN NBA [Электронный ресурс]. — Новости и обзоры НБА (справочно о медиа-формате представления статистики). — Режим доступа: <https://www.espn.com/nba/> (дата обращения: 29.03.2026).
7. Коннолли, Т. Базы данных. Проектирование, реализация и сопровождение. Теория и практика / Т. Коннолли, К. Бегг ; пер. с англ. — 3-е изд. — М. : Вильямс, 2003. — 1440 с.
8. Chen, P. P. The Entity-Relationship Model — Toward a Unified View of Data / P. P. Chen // ACM Transactions on Database Systems. — 1976. — Vol. 1, № 1. — P. 9–36.
9. Дейт, К. Дж. Введение в системы баз данных / К. Дж. Дейт ; пер. с англ. — 8-е изд. — М. : Вильямс, 2006. — 1328 с.
10. Redis Documentation [Электронный ресурс]. — Документация in-memory хранилища Redis. — Режим доступа: <https://redis.io/docs/> (дата обращения: 29.03.2026).
11. PostgreSQL 15 Documentation [Электронный ресурс]. — Официальная документация СУБД PostgreSQL. — Режим доступа: <https://www.postgresql.org/docs/15/> (дата обращения: 29.03.2026).

12. Python 3 Documentation [Электронный ресурс]. — Официальная документация языка программирования Python. — Режим доступа: <https://docs.python.org/3/> (дата обращения: 29.03.2026).
13. FastAPI [Электронный ресурс]. — Официальная документация веб-фреймворка FastAPI. — Режим доступа: <https://fastapi.tiangolo.com/> (дата обращения: 29.03.2026).
14. React Documentation [Электронный ресурс]. — Официальная документация библиотеки React. — Режим доступа: <https://react.dev/> (дата обращения: 29.03.2026).